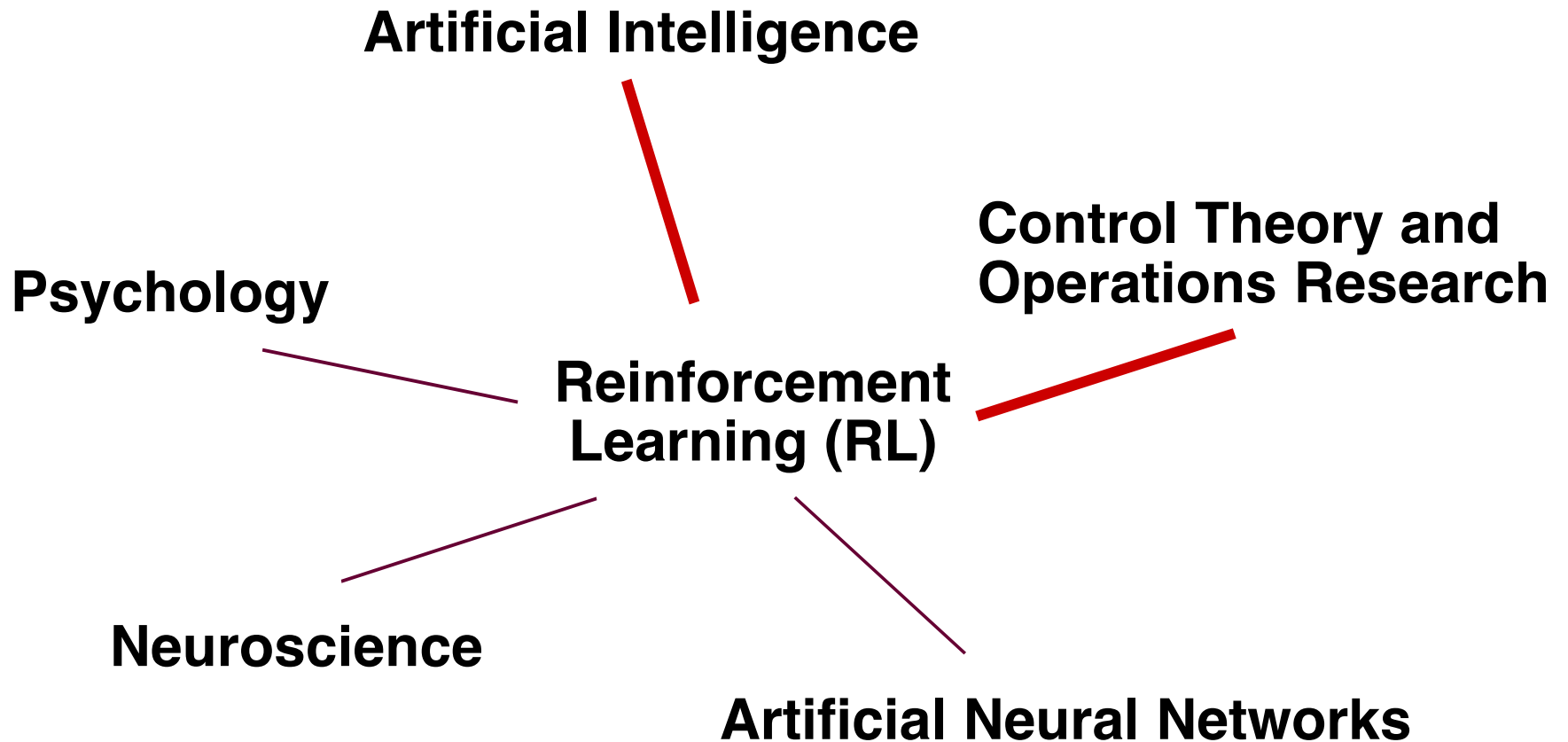


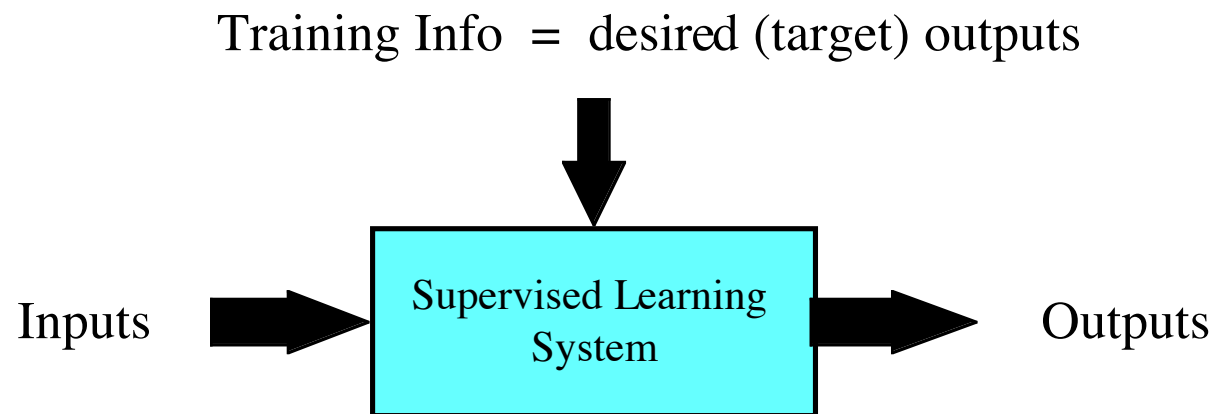
Chapter 1: Introduction



What is Reinforcement Learning?

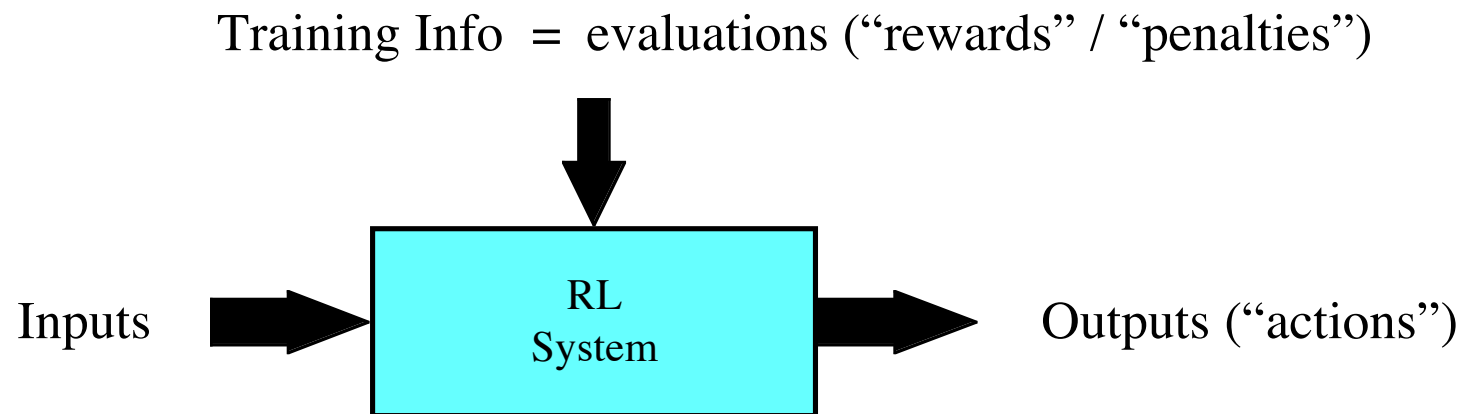
- ❑ Learning from interaction
- ❑ Goal-oriented learning
- ❑ Learning about, from, and while interacting with an external environment
- ❑ Learning what to do—how to map situations to actions—so as to maximize a numerical reward signal

Supervised Learning



$$\text{Error} = (\text{target output} - \text{actual output})$$

Reinforcement Learning



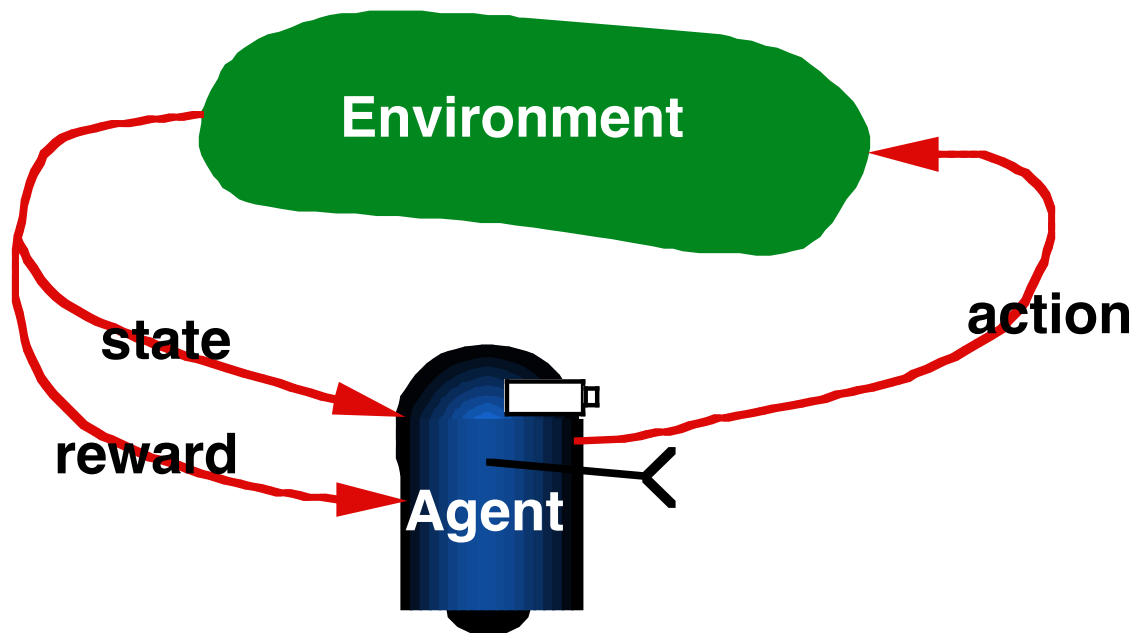
Objective: get as much reward as possible

Key Features of RL

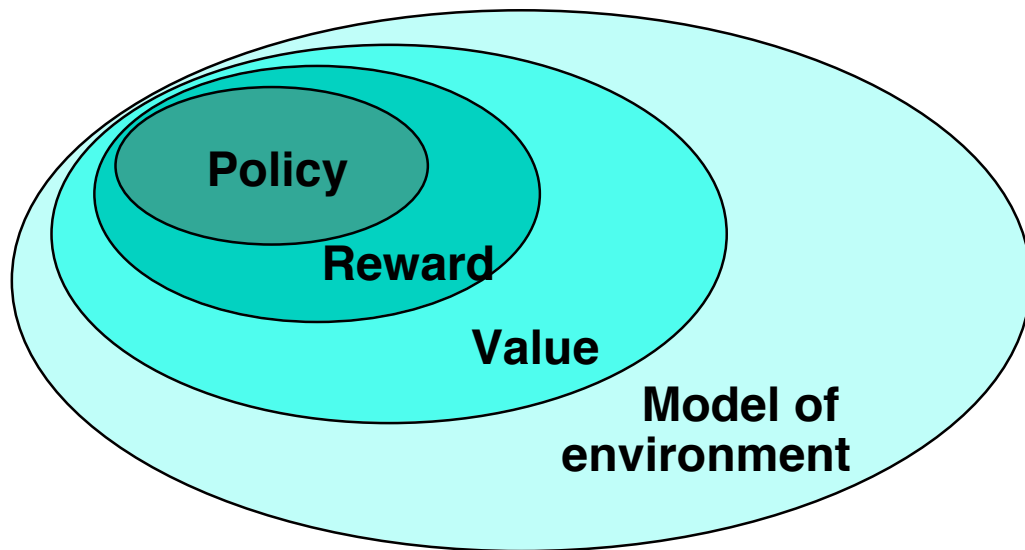
- ❑ Learner is not told which actions to take
- ❑ Trial-and-Error search
- ❑ Possibility of delayed reward
 - Sacrifice short-term gains for greater long-term gains
- ❑ The need to *explore* and *exploit*
- ❑ Considers the whole problem of a goal-directed agent interacting with an uncertain environment

Complete Agent

- ❑ Temporally situated
- ❑ Continual learning and planning
- ❑ Object is to *affect* the environment
- ❑ Environment is stochastic and uncertain



Elements of RL



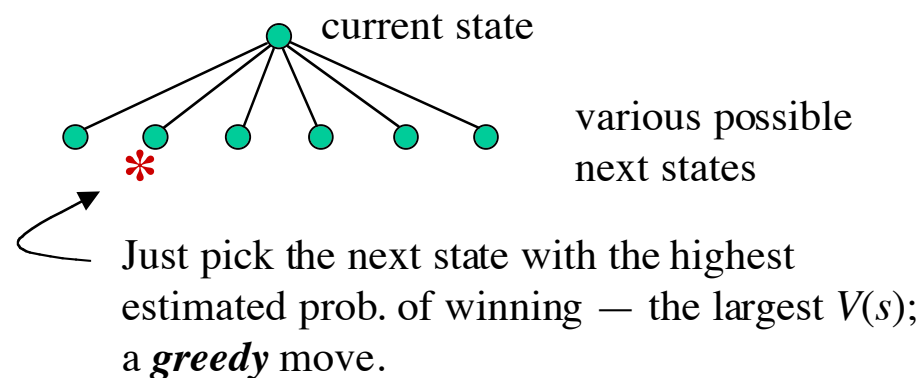
- ❑ **Policy:** what to do
- ❑ **Reward:** what is good
- ❑ **Value:** what is good because it *predicts* reward
- ❑ **Model:** what follows what

An RL Approach to Tic-Tac-Toe

1. Make a table with one entry per state:

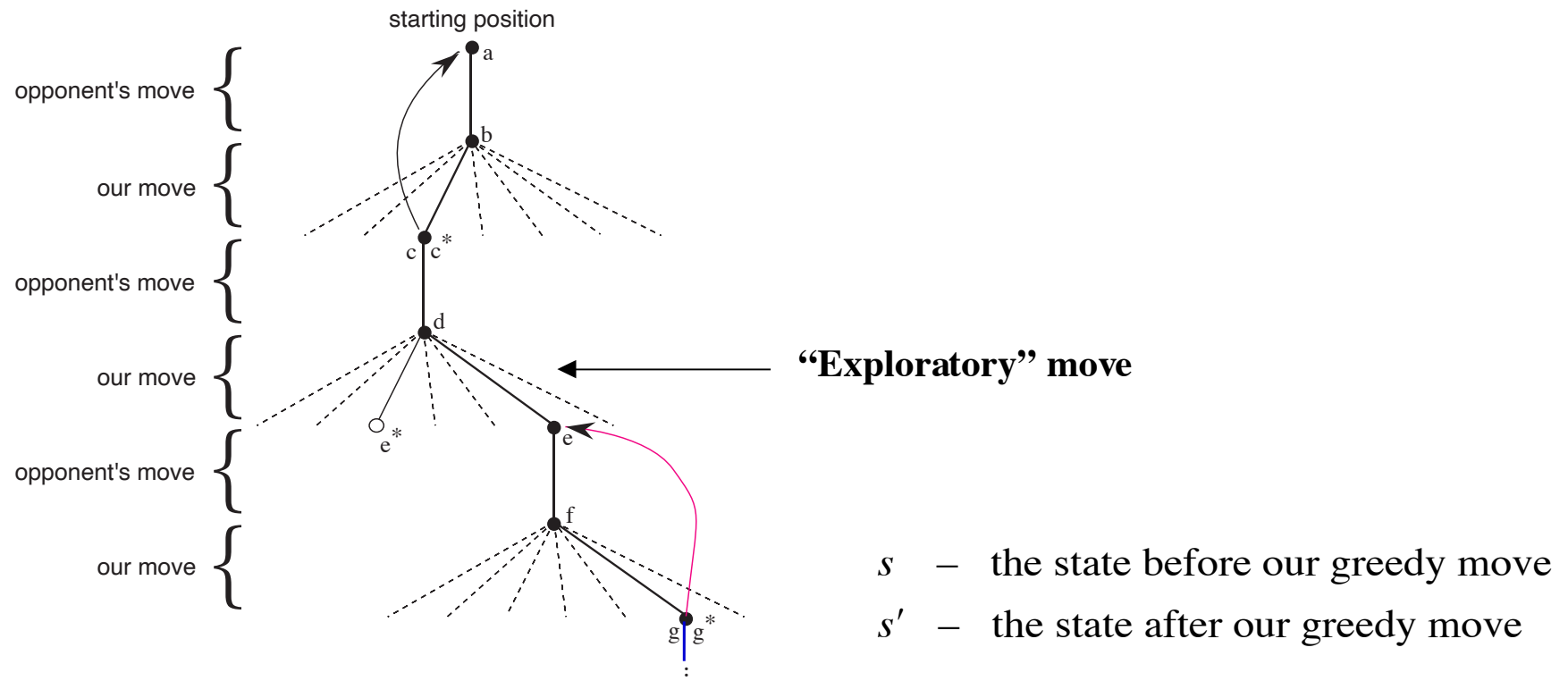
State	$V(s)$ – estimated probability of winning	
$\begin{array}{ c c c }\hline & & \\ \hline & & \\ \hline & & \\ \hline\end{array}$	.5	?
$\begin{array}{ c c c }\hline x & & \\ \hline & & \\ \hline & & \\ \hline\end{array}$	.5	?
⋮	⋮	
$\begin{array}{ c c c }\hline x & x & x \\ \hline o & & \\ \hline & & \\ \hline\end{array}$	1	win
⋮	⋮	
$\begin{array}{ c c c }\hline & x & o \\ \hline x & & o \\ \hline & & o \\ \hline\end{array}$	0	loss
⋮	⋮	
$\begin{array}{ c c c }\hline o & x & o \\ \hline o & x & x \\ \hline x & o & o \\ \hline\end{array}$	0	draw

2. Now play lots of games.
To pick our moves,
look ahead one step:



But 10% of the time pick a move at random;
an **exploratory move**.

RL Learning Rule for Tic-Tac-Toe

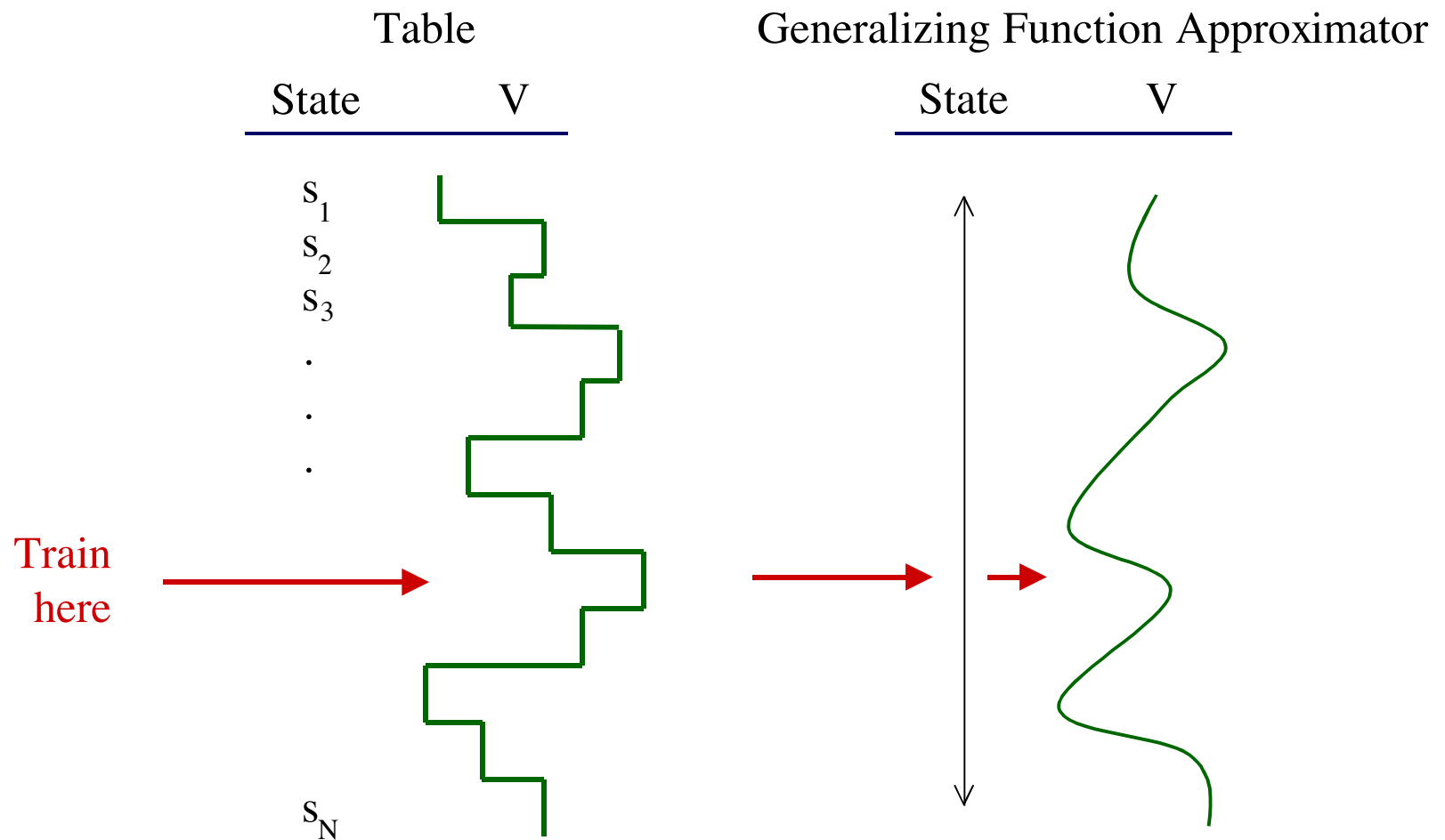


We increment each $V(s)$ toward $V(s')$ – a **backup** :

$$V(s) \leftarrow V(s) + \alpha[V(s') - V(s)]$$

a small positive fraction, e.g., $\alpha = .1$
the **step - size parameter**

e.g. Generalization



How is Tic-Tac-Toe Too Easy?

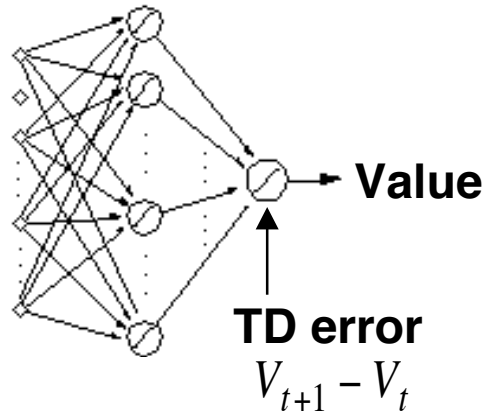
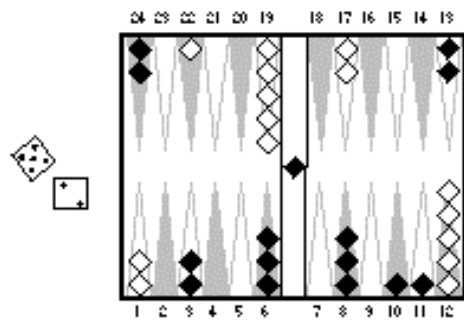
- ❑ Finite, small number of states
- ❑ One-step look-ahead is always possible
- ❑ State completely observable
- ❑ . . .

Some Notable RL Applications

- ❑ **TD-Gammon:** Tesauro
 - world's best backgammon program
- ❑ **Elevator Control:** Crites & Barto
 - high performance down-peak elevator controller
- ❑ **Inventory Management:** Van Roy, Bertsekas, Lee&Tsitsiklis
 - 10–15% improvement over industry standard methods
- ❑ **Dynamic Channel Assignment:** Singh & Bertsekas, Nie & Haykin
 - high performance assignment of radio channels to mobile telephone calls

TD-Gammon

Tesauro, 1992–1995



Action selection
by 2–3 ply search

Start with a random network

Play very many games against self

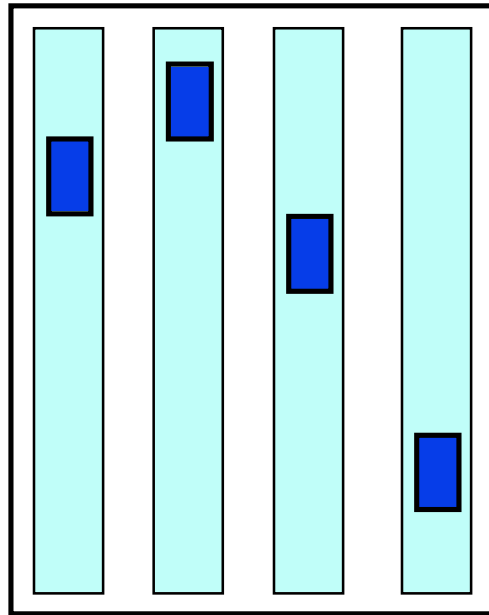
Learn a value function from this simulated experience

This produces arguably the best player in the world

Elevator Dispatching

Crites and Barto, 1996

10 floors, 4 elevator cars



STATES: button states; positions, directions, and motion states of cars; passengers in cars & in halls

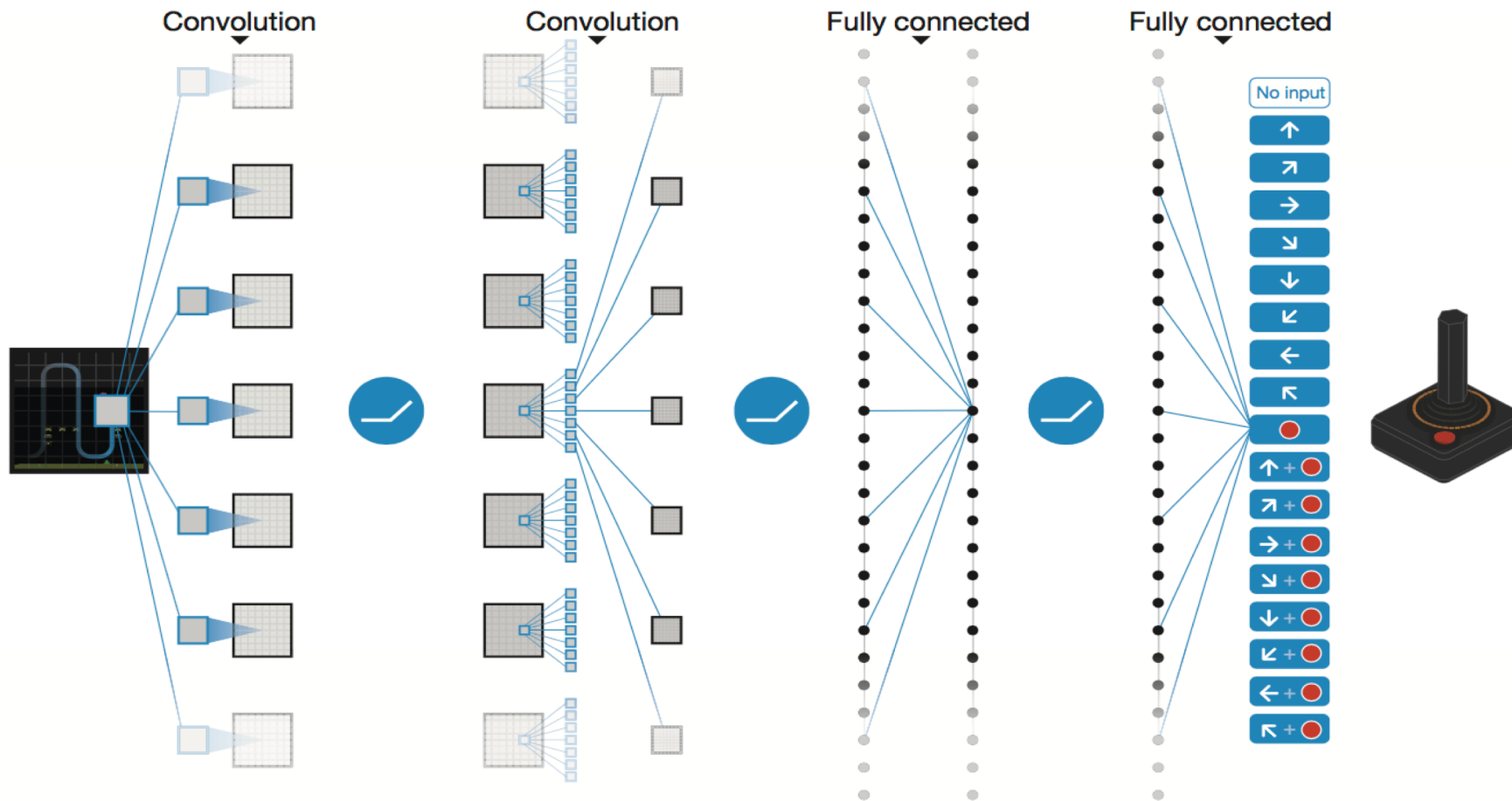
ACTIONS: stop at, or go by, next floor

REWARDS: roughly, -1 per time step for each person waiting

Conservatively about 10^{22} states

DQN

NATURE
VOL 518
26 FEBRUARY 2015



<https://deepmind.com/research/dqn/>

CODE -> <https://sites.google.com/a/deepmind.com/dqn/>

ARTICLE



AlphaGo Zero

doi:10.1038/nature24270

NATURE
VOL 550
19 OCTOBER 2017

Mastering the game of Go without human knowledge

David Silver^{1*}, Julian Schrittwieser^{1*}, Karen Simonyan^{1*}, Ioannis Antonoglou¹, Aja Huang¹, Arthur Guez¹, Thomas Hubert¹, Lucas Baker¹, Matthew Lai¹, Adrian Bolton¹, Yutian Chen¹, Timothy Lillicrap¹, Fan Hui¹, Laurent Sifre¹, George van den Driessche¹, Thore Graepel¹ & Demis Hassabis¹

A long-standing goal of artificial intelligence is an algorithm that learns, *tabula rasa*, superhuman proficiency in challenging domains. Recently, AlphaGo became the first program to defeat a world champion in the game of Go. The tree search in AlphaGo evaluated positions and selected moves using deep neural networks. These neural networks were trained by supervised learning from human expert moves, and by reinforcement learning from self-play. Here we introduce an algorithm based solely on reinforcement learning, without human data, guidance or domain knowledge beyond game rules. AlphaGo becomes its own teacher: a neural network is trained to predict AlphaGo's own move selections and also the winner of AlphaGo's games. This neural network improves the strength of the tree search, resulting in higher quality move selection and stronger self-play in the next iteration. Starting *tabula rasa*, our new program AlphaGo Zero achieved superhuman performance, winning 100–0 against the previously published, champion-defeating AlphaGo.

<https://deepmind.com/blog/alphago-zero-learning-scratch/>